Estratégia Concursos

Aula 20

DataPrev (Analista de TI - Desenvolvimento de Software) Passo Estratégico de Conhecimentos Específicos - 2024 (Pós-Edital)

Autor:

09 de Setembro de 2024

11225354498 - Maria Vitória Gonçalves Vieira

Gestão de Configuração e Mudança

Sumário

Conteúdo 1
Glossário de termos 2
Roteiro de revisão 4
Introdução 4
Planejamento e Implementação de GCM 8
Controle de Versão 10
Gerenciamento de Mudanças 13
Auditoria e Controle de GCM 17
Git 17
Github 23
Gitlab 25
Aposta estratégica 26
Questões Estratégicas 27
Questionário de revisão e aperfeiçoamento 31
Perguntas 31
Perguntas e Respostas 32
Lista de Questões Estratégicas 34

Conteúdo

Gestão de Configuração e Mudanças (GCM). Definições. Objetivos. Elementos. Visão geral do processo de GCM. Planejamento e implementação de GCM. Ferramentas. Processos. Melhores Práticas. Controle de versão. Gerenciamento de mudanças. Auditora e Controle de GCM. Git. GitHub. GitLab.

Análise Estatística

Inicialmente, convém destacar o percentual de incidência do assunto, dentro da disciplina Engenharia de Software em concursos/cargos similares. Quanto maior o percentual de cobrança de um dado assunto, maior sua importância.

Obs.: *um mesmo assunto pode ser classificado em mais de um tópico devido à multidisciplinaridade de conteúdo.*

Assunto	Relevância na disciplina em concursos similares

11225354498 - Maria Vitória Gonçalves Vieira

UML	**11.0 %**
Processos de Software - Desenvolvimento Ágil	**8.4 %**
Engenharia de Requisitos	**8.2 %**
Teste de Software	**6.7 %**
Métricas de Software	**5.1 %**
Desenvolvimento de Software	**4.6 %**
Inteligencia Artificial	**4.6 %**
Processos de Software	**4.6 %**
Orientação a Objetos	**4.5 %**
Gestão de Projetos em Engenharia de Software	**3.6 %**
Qualidade de Software	**3.4 %**
Metodologia de desenvolvimento de software	**2.5 %**
Gerência de Configuração	**1.9 %**
Geoprocessamento em Engenharia de Software	**1.6 %**
Prototipação	**1.2 %**
Conceitos Básicos em Engenharia de Software	**0.7 %**
Análise Estruturada	**0.6 %**
Ferramentas CASE	**0.4 %**
Ferramentas de Desenvolvimento de Software	**0.4 %**
Software livre	**0.3 %**
Manutenção de Software	**0.1 %**
Análise Essencial	**0.1 %**
Web 2.0	**0.1 %**
Portal Web	**0.1 %**
Refatoração	**0.1 %**
Engenharia da Informação	**0.1 %**

Glossário de termos

Faremos uma lista de termos que são relevantes ao entendimento do assunto desta aula. Caso tenha alguma dúvida durante a leitura, esta seção pode lhe ajudar a esclarecer.

GCM (Gestão de Configuração e Mudanças): Prática de gerenciar mudanças em sistemas de maneira sistemática para que os sistemas mantenham sua integridade ao longo do tempo.

Item de Configuração: Um agregado de hardware, software ou ambos, que é designado para gerenciamento de configuração e tratado como uma única entidade no processo de GCM.

11225354498 - Maria Vitória Gonçalves Vieira

Controle de Versão: Processo de gerenciar diferentes versões de artefatos de software que são produzidos durante o ciclo de vida de um software.
Controle de Mudança: O processo de gerenciar e controlar alterações nos requisitos de um sistema ou projeto de software.
Baseline: Estado estável e aprovado de um sistema em um determinado momento do ciclo de vida do projeto.
Auditoria de Configuração: Uma inspeção e verificação independente da conformidade dos produtos de trabalho de software e atividades com os requisitos, procedimentos e padrões especificados.
Integração Contínua (CI): Prática de desenvolvimento que envolve integrar o trabalho de um desenvolvedor com o do resto da equipe de desenvolvimento com frequência, geralmente várias vezes ao dia.
Git: Um sistema de controle de versão distribuído gratuito e de código aberto que é projetado para lidar com tudo, desde pequenos a grandes projetos com velocidade e eficiência.
SVN (Subversion): Uma ferramenta de controle de versão de código aberto.
Mercurial: Outra ferramenta de controle de versão distribuído semelhante ao Git, mas com uma interface mais simples.
Jira: Um produto de software de rastreamento de problemas desenvolvido pela Atlassian que permite o gerenciamento de bugs e a atribuição de tarefas.
Bugzilla: Uma ferramenta de rastreamento de bugs baseada na web que permite que os desenvolvedores acompanhem os bugs pendentes e o progresso das tarefas.
Redmine: Uma aplicação web de gerenciamento de projetos flexível que fornece rastreamento de problemas, entre outros recursos.
Jenkins: Uma ferramenta de código aberto para a implementação de Integração Contínua e Entrega Contínua.
Travis CI: Um serviço de Integração Contínua hospedado usado para construir e testar projetos de software hospedados no GitHub.
CircleCI: Outra plataforma de Integração Contínua que permite que os desenvolvedores construam, testem e implementem aplicações.
GitHub Actions: Uma solução de CI/CD fornecida pelo GitHub que permite automatizar fluxos de trabalho.
Ansible: Uma plataforma de software de código aberto para configuração e gerenciamento de computadores.
Chef: Uma ferramenta para automatizar a configuração, implantação e gerenciamento de infraestrutura.
Puppet: Uma ferramenta de gerenciamento de configuração que permite definir o estado de sua infraestrutura como código.
Terraform: Uma ferramenta de infraestrutura como código para construir, mudar e versionar a infraestrutura de maneira segura e eficiente.



11225354498 - Maria Vitória Gonçalves Vieira

Artifactory: Um gerenciador de repositório binário universal que pode armazenar artefatos de qualquer formato.
Nexus: Uma plataforma de gerenciamento de repositório que ajuda a gerenciar e controlar o fluxo de artefatos em sua organização.
Modelo Centralizado: Um tipo de sistema de controle de versão em que todos os usuários se conectam a um servidor central que contém todos os arquivos de versão.
Modelo Distribuído: Um tipo de sistema de controle de versão em que cada usuário tem uma cópia completa do repositório, permitindo o trabalho offline e não dependente de um único servidor central.
GitHub: Uma plataforma de hospedagem de código-fonte que fornece hospedagem para repositórios Git, bem como recursos de colaboração como bug tracking, gerenciamento de tarefas e wikis para cada projeto.
GitLab: Uma plataforma completa de DevOps de código aberto que permite aos profissionais colaborar em projetos de software.
Pull Request: Uma proposta de alteração em um projeto de software que está hospedado em um repositório. Uma vez que a alteração é aprovada, ela pode ser mergeada no projeto.
Issue: Problemas ou melhorias sugeridas para um projeto de software que são rastreados e gerenciados no GitHub.
Fork: Uma cópia de um repositório que reside na conta do usuário e permite experimentar alterações sem afetar o projeto original.

ROTEIRO DE REVISÃO

A ideia desta seção é apresentar um roteiro para que você realize uma revisão completa do assunto e, ao mesmo tempo, destacar aspectos do conteúdo que merecem atenção.

Introdução

Gestão de Configuração e Mudanças (GCM) é um conjunto de processos, técnicas e ferramentas que ajudam a gerenciar e controlar as alterações feitas durante o ciclo de vida do desenvolvimento de software. Isso inclui rastreamento de mudanças, manutenção de versões e variantes do software, e assegurar que as mudanças e configurações feitas não introduzam erros ou problemas adicionais.
Os principais objetivos da GCM são:

11225354498 - Maria Vitória Gonçalves Vieira

- Garantir a consistência do software, mantendo um registro das versões e variações do software e assegurando que as mudanças sejam devidamente implementadas e rastreadas.
- Facilitar a comunicação e a coordenação entre a equipe de desenvolvimento, permitindo que todos estejam cientes e compreendam as mudanças e configurações feitas.
- Prevenir a introdução de erros ou problemas devido a alterações não controladas ou mal gerenciadas.
- Reduzir os riscos associados com o desenvolvimento de software, através do gerenciamento efetivo e cuidadoso das mudanças e configurações.

A GCM é crucial na engenharia de software devido a sua capacidade de gerenciar e controlar as mudanças que ocorrem durante o desenvolvimento de software. Ela ajuda a evitar a deterioração do projeto e assegura que o software final atenda às especificações e requisitos definidos.

Por exemplo, sem GCM, alterações no código podem ser feitas sem o devido rastreamento, o que pode levar a erros e problemas difíceis de serem identificados e corrigidos. Além disso, a falta de um registro das versões do software pode levar a confusão e a problemas de compatibilidade.

Gestão de Configuração e Mudanças é particularmente importante em projetos grandes e complexos, onde múltiplas equipes ou indivíduos podem estar trabalhando em diferentes partes do software simultaneamente. Ela permite que todos estejam cientes das mudanças e configurações feitas, facilitando a comunicação e a coordenação entre as equipes e ajudando a garantir que todos estejam trabalhando na mesma versão do software.

Um exemplo de GCM em ação pode ser visto em projetos de código aberto, onde muitos desenvolvedores de todo o mundo contribuem com o código. Ferramentas como Git são usadas para gerenciar e rastrear as mudanças feitas por cada contribuidor, garantindo que ninguém está trabalhando em uma versão desatualizada do software e que todas as mudanças são devidamente registradas e revisadas.

Outro exemplo pode ser encontrado em empresas de software de um modo geral, onde diferentes equipes podem estar trabalhando em diferentes partes de um produto de software ao mesmo tempo. A GCM permite que essas equipes coordenem seus esforços, garantindo que as alterações feitas por uma equipe não interfiram no trabalho de outra.

Itens de Configuração

Itens de configuração são partes individuais de um sistema gerenciadas durante o ciclo de vida de um projeto. Esses itens podem variar muito, dependendo do projeto específico, mas geralmente incluem o código fonte, a documentação, os dados de

11225354498 - Maria Vitória Gonçalves Vieira

configuração, os artefatos de design, as ferramentas de construção, os binários e releases, e os testes e dados de teste.

A Gestão de Configuração e Mudanças é essencial para controlar a evolução desses itens e garantir que todas as alterações sejam devidamente rastreadas e documentadas. Isso mantém a consistência do sistema e reduz o risco de problemas devido a mudanças não controladas.

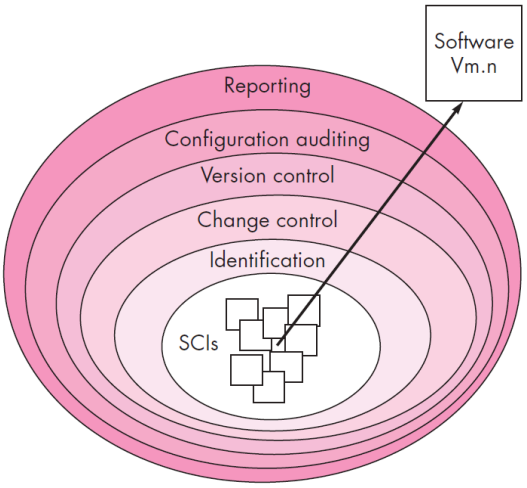
Veja exemplos de itens de configuração:

- **Código Fonte:** O código-fonte dos programas e bibliotecas constitui um dos principais elementos de configuração em um projeto de software. Cada arquivo de código-fonte é um item de configuração que pode ser versionado e gerenciado individualmente.
- **Artefatos de Design:** Documentos de design, diagramas UML, e outras representações do design do software também são considerados elementos de configuração. Eles ajudam a definir a arquitetura do software e orientam o desenvolvimento.
- **Documentação:** Isso pode incluir a documentação do usuário, documentação técnica, especificações de requisitos, planos de teste e outros documentos relacionados ao software. Esses documentos são essenciais para a operação, manutenção e uso do software.
- **Dados de Configuração:** Inclui todos os arquivos e dados que controlam a operação do software, como arquivos de configuração, scripts de banco de dados, arquivos de propriedades e outros dados que definem o ambiente em que o software opera.
- **Ferramentas de Construção:** Scripts de build, arquivos Makefile, configurações de ferramentas de compilação e outros elementos usados na construção do software também são considerados elementos de configuração.
- **Binários e Releases:** Versões compiladas do software, conhecidas como binários, e versões oficiais do software, conhecidas como releases, também são elementos de configuração importantes.
- **Testes e Dados de Teste:** Códigos de teste, scripts de teste e conjuntos de dados usados para testar o software também são considerados elementos de configuração.

Processo de GCM

O Processo de Gerenciamento de Configuração e Mudança define uma série de tarefas que têm quatro objetivos primários: (1) identificar todos os itens que coletivamente definem a configuração do software, (2) gerenciar mudanças em um ou mais itens, (3) facilitar a construção de diferentes versões de uma aplicação, e (4) assegurar que a qualidade seja mantida à medida que a configuração evolui. Um processo que alcança esses objetivos não necessita ser burocrático ou pesado, mas deve levar à definição de cinco atividades principais:

11225354498 - Maria Vitória Gonçalves Vieira



1. Identificação de Itens de Configuração:
A identificação é o primeiro passo na GCM. Aqui, os itens de configuração que precisam ser gerenciados são identificados. Inclui código-fonte, documentação, artefatos de design, dados de configuração, ferramentas de construção, binários, testes e dados de teste. Cada item de configuração é então rotulado de forma única para garantir a rastreabilidade.
2. Controle de Versão:
O controle de versão é a prática de gerenciar diferentes versões de itens de configuração. Isso envolve o rastreamento e o controle de alterações nos itens de configuração ao longo do tempo, de forma que as versões específicas possam ser recuperadas conforme necessário. Muitas vezes é realizado usando ferramentas de controle de versão como Git ou SVN.
3. Controle de Mudança:
O controle de mudança envolve o gerenciamento das alterações nos itens de configuração. Inclui a avaliação e aprovação de propostas de mudança, a implementação de mudanças aprovadas e a revisão das mudanças após a implementação. O objetivo é minimizar o impacto das mudanças na qualidade do software e na continuidade do projeto.
4. Auditoria de Configuração:
A auditoria de configuração é o processo de verificar se a configuração atual do software está de acordo com as informações documentadas sobre a configuração. Compreende a verificação da consistência e completude dos itens de configuração e o cumprimento das políticas e procedimentos de GCM.
5. Relatório de Status:
O relatório de status envolve o fornecimento de informações atualizadas sobre o estado dos itens de configuração e as mudanças que foram feitas. Abrange

11225354498 - Maria Vitória Gonçalves Vieira

informações sobre quais mudanças foram implementadas, quais estão pendentes e quais foram rejeitadas, além do status atual de cada item de configuração.

Planejamento e Implementação de GCM

O desenvolvimento do Plano de Gestão de Configuração e Mudanças é uma etapa importante no processo de GCM. Este plano descreve as políticas, procedimentos e responsabilidades para o gerenciamento de itens de configuração e mudanças durante o ciclo de vida do projeto. O Plano de GCM geralmente inclui os seguintes componentes:

- Introdução: É o ponto de partida do plano de GCM. Deve incluir uma breve descrição do projeto e o propósito do plano de GCM.
- Definição de Itens de Configuração: Esta seção descreve quais elementos do projeto serão tratados como itens de configuração. Inclui código-fonte, documentação, dados de configuração, ferramentas de compilação, binários e testes, entre outros.
- Processos de GCM: Descreve os processos específicos que serão usados para gerenciar os itens de configuração e controlar as mudanças. Abrange processos de identificação, controle de versão, controle de mudança, auditoria de configuração e relatório de status.
- Responsabilidades e Recursos: Nesta seção, as responsabilidades de diferentes partes interessadas no processo de GCM são definidas. Compreende desenvolvedores, gerentes de projeto, equipes de qualidade, entre outros. Além disso, os recursos necessários para implementar o plano de GCM são identificados.
- Ferramentas de GCM: Esta parte do plano descreve quais ferramentas serão usadas para suportar os processos de GCM. Pode incluir ferramentas de controle de versão como Git, ferramentas de rastreamento de problemas como Jira, e outras ferramentas de GCM.
- Procedimentos de Auditoria e Revisão: O plano deve definir como as auditorias de configuração e as revisões serão conduzidas para garantir que o plano de GCM está sendo seguido e que os itens de configuração estão sendo devidamente gerenciados.
- Treinamento e Educação: Finalmente, o plano deve descrever qualquer treinamento ou educação necessária para garantir que todos os envolvidos no projeto compreendam e possam seguir o plano de GCM.

Seleção de Ferramentas

Para selecionar ferramentas de GCM, primeiro é preciso entender as necessidades do seu projeto. Cada projeto tem requisitos únicos, e a ferramenta que funcionará melhor para você depende do tamanho da sua equipe, da complexidade do seu projeto e das tecnologias que você está usando.

11225354498 - Maria Vitória Gonçalves Vieira

A tabela a seguir resume as principais categorias de ferramentas, tanto de GCM como de outros processos relacionados, e alguns exemplos de ferramentas disponíveis:

Categoria	Objetivo	Ferramentas
Controle de Versão	Gerenciar e rastrear diferentes versões de itens de configuração ao longo do tempo.	Git, SVN, Mercurial
Rastreamento de Mudanças	Gerenciar e rastrear solicitações de mudanças, bugs e melhorias.	Jira, Bugzilla, Redmine
Build e Integração Contínua	Automatizar o processo de compilação, teste e implantação de software.	Jenkins, Travis CI, CircleCI, GitHub Actions
Gerenciamento de Configuração	Gerenciar e automatizar a configuração de sistemas e infraestruturas de software.	Ansible, Chef, Puppet, Terraform
Gestão de Repositórios de Artefatos	Gerenciar e armazenar artefatos de software e dependências em um local centralizado.	Artifactory, Nexus

Melhores Práticas

Implementar a Gestão de Configuração e Mudanças (GCM) de maneira eficaz requer a adoção de uma série de melhores práticas. Aqui estão algumas das mais importantes:

Planejamento detalhado: Comece com um plano sólido de GCM que detalhe as políticas, procedimentos e responsabilidades para gerenciar os itens de configuração e controlar as mudanças. Este plano deve ser revisado e atualizado regularmente à medida que o projeto evolui.

Identificação clara de Itens de Configuração: Os itens de configuração devem ser claramente identificados e rotulados. Isso facilita a rastreabilidade e garante que todas as alterações possam ser devidamente gerenciadas.

Uso de ferramentas de GCM: As ferramentas de GCM facilitam o gerenciamento de itens de configuração e o controle de mudanças. Ao escolher ferramentas, é importante considerar suas necessidades específicas e garantir que as ferramentas se integrem bem umas com as outras.

11225354498 - Maria Vitória Gonçalves Vieira

Controle rigoroso de mudanças: Todas as mudanças nos itens de configuração devem passar por um processo de controle de mudanças. Isso inclui a avaliação e aprovação de propostas de mudança, a implementação de mudanças aprovadas e a revisão das mudanças após a implementação.

Rastreabilidade: Assegure que cada alteração possa ser rastreada desde a solicitação inicial até a implementação final. Isso não só ajuda na resolução de problemas, mas também é útil para auditorias e conformidade.

Auditorias regulares: As auditorias de configuração devem ser realizadas regularmente para verificar se a configuração atual do software está de acordo com as informações documentadas sobre a configuração.

Comunicação eficaz: As informações sobre os itens de configuração e as mudanças devem ser comunicadas eficazmente a todas as partes interessadas. Isso pode ser facilitado através de relatórios de status regulares.

Treinamento e Educação: Garanta que todos os envolvidos no projeto compreendam e possam seguir o plano de GCM. Isso pode exigir treinamento ou educação específicos.

Controle de Versão

Controle de versão, também conhecido como gerenciamento de versões ou controle de revisão, é uma prática em engenharia de software que rastreia e fornece controle sobre as mudanças nos códigos fonte e outros tipos de informação de trabalho. Esse processo é importante para permitir a colaboração efetiva entre as equipes de desenvolvimento e para gerenciar as diferentes versões de um produto de software ao longo do tempo.

Em resumo, o controle de versão:

- Facilita a colaboração: Ele permite que várias pessoas trabalhem no mesmo projeto sem se sobreporem, pois cada uma pode trabalhar em diferentes versões do mesmo arquivo e, em seguida, as alterações podem ser mescladas.
- Mantém um histórico de mudanças: Todas as alterações feitas nos arquivos são rastreadas junto com detalhes como quem fez a mudança, por que a mudança foi feita e quando a mudança foi feita. Isso facilita a detecção de quando e onde os erros foram introduzidos.

11225354498 - Maria Vitória Gonçalves Vieira

- **Permite a recuperação de versões anteriores:** Se algo der errado com a versão atual, é possível voltar para uma versão anterior que estava funcionando corretamente.
- **Facilita a gestão de releases:** O controle de versão permite que você mantenha versões estáveis do seu software (por exemplo, para lançamento ao público) separadas das versões em desenvolvimento.

Existem dois principais tipos de sistemas de controle de versão: **centralizado** (como SVN) e **distribuído** (como Git).

Modelo Centralizado

Exemplos populares: Subversion (SVN), Perforce

Neste modelo, existe um único servidor central que contém todas as versões dos arquivos do projeto. Os desenvolvedores podem fazer check-out de arquivos deste servidor central, mas geralmente apenas um desenvolvedor por vez pode fazer mudanças em um arquivo específico. Após as alterações, eles fazem check-in das atualizações de volta ao servidor central.

Vantagens:

- Simplicidade: Este modelo é mais fácil de entender para novos usuários, principalmente se eles estiverem acostumados a trabalhar com sistemas de arquivos centralizados.
- Controle de acesso: O modelo centralizado permite um controle mais granular sobre quem pode fazer o que, o que pode ser benéfico em organizações maiores ou mais hierárquicas.

Desvantagens:

- Único ponto de falha: Se o servidor central falhar e não houver backup adequado, todo o histórico do projeto pode ser perdido.
- Dependência da rede: Os desenvolvedores precisam estar conectados à rede para fazer check-in ou check-out de arquivos, o que pode ser inconveniente se a conexão de rede for lenta ou instável.

Modelo Distribuído

11225354498 - Maria Vitória Gonçalves Vieira

Exemplos populares: Git, Mercurial

Neste modelo, cada desenvolvedor tem uma cópia local do repositório inteiro, incluindo todo o histórico de versões. Isso significa que os desenvolvedores podem fazer muitas operações (como commits e diffs) localmente, sem necessidade de conexão de rede. Quando os desenvolvedores querem compartilhar suas alterações, eles as "empurram" (push) para um repositório compartilhado.

Vantagens:

- Desempenho: Como muitas operações são locais, elas geralmente são mais rápidas do que em um sistema centralizado.
- Flexibilidade: Os desenvolvedores podem trabalhar e fazer commits em seu próprio ritmo, sem ter que se preocupar em interferir no trabalho de outras pessoas.

Desvantagens:

- Complexidade: Os sistemas de controle de versão distribuídos têm uma curva de aprendizado mais acentuada, pois há muitos conceitos novos que não são necessários em um sistema centralizado.
- Falta de controle de acesso granular: É mais difícil controlar quem pode fazer o que em um sistema distribuído, o que pode ser um problema em organizações maiores ou mais hierárquicas.

Baseline

Baseline é um termo amplamente utilizado em gestão de configuração que se refere a um marco ou ponto de referência estável no desenvolvimento de um projeto. Pode ser entendida como uma "fotografia" ou um estado estável e aprovado de um sistema em um determinado momento do ciclo de vida do projeto.

Cada baseline é uma versão especificada de um sistema durante o seu ciclo de vida e serve como um ponto de partida para futuras mudanças. Quando as mudanças são feitas a partir de uma baseline, essas mudanças são feitas em relação a essa baseline. Desta forma, as baselines desempenham um papel importante na gestão de mudanças, pois permitem que os gerentes de projeto tenham um controle claro sobre as alterações feitas.

Existem três tipos comuns de baselines em gestão de configuração:

11225354498 - Maria Vitória Gonçalves Vieira

1. Baseline Funcional: Estabelecida após a fase de requisitos. Reflete todos os requisitos do cliente e é usada como uma base para o desenvolvimento de software.
2. Baseline de Alocação: Estabelecida após a fase de projeto e inclui uma arquitetura de software aprovada que detalha a estrutura e as relações dos componentes do sistema.
3. Baseline de Produto: Estabelecida após a fase de implementação e inclui o código-fonte final, dados, documentação do usuário e de sistema, e outros entregáveis necessários.

Gerenciamento de Mudanças

Gestão de Mudanças é uma disciplina estruturada que visa garantir que as alterações sejam implementadas de maneira suave e eficiente para minimizar o impacto na estabilidade e na rotina da organização.

Em engenharia de software, a gestão de mudanças geralmente se refere à gestão sistemática de todas as mudanças feitas no sistema de software. Isso inclui não apenas alterações no próprio código, mas também alterações na arquitetura do sistema, no hardware subjacente, na documentação e em outros aspectos do sistema.

O objetivo da gestão de mudanças em projetos de software é garantir que todas as mudanças sejam rastreadas e controladas, para evitar problemas que podem surgir de mudanças não controladas, como a introdução de novos defeitos ou a reaparecimento de defeitos antigos.

O processo envolve várias etapas, tais como:

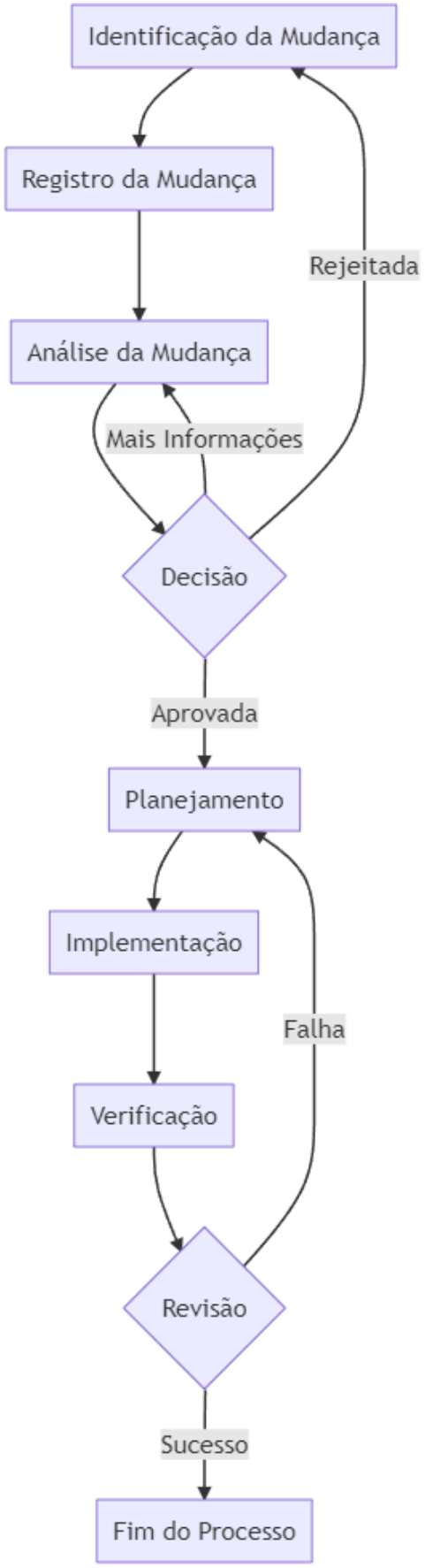
1. Identificação da Mudança: Uma alteração é identificada, geralmente por uma parte interessada no projeto, como um membro da equipe, cliente ou usuário.
2. Registro da Mudança: A mudança é documentada em um sistema de rastreamento de mudanças. Detalhes como a descrição da mudança, o motivo da mudança, e a pessoa que a identificou são incluídos.
3. Análise da Mudança: A mudança é avaliada em termos de seu impacto nos negócios, custos, tempo, recursos e riscos envolvidos.
4. Decisão: O Comitê de Controle de Mudanças (CCM) toma uma decisão sobre a mudança - seja para aprovar, rejeitar ou pedir mais informações.
5. Planejamento: Se a mudança for aprovada, um plano é desenvolvido para implementá-la. O plano inclui tarefas, recursos, prazos e medidas de sucesso.

11225354498 - Maria Vitória Gonçalves Vieira

6. Implementação: A mudança é implementada conforme planejado.
7. Verificação: Após a implementação, a mudança é testada e verificada para garantir que tenha sido implementada corretamente e esteja funcionando conforme esperado.
8. Revisão: A eficácia da mudança é avaliada. Se a mudança não atender aos critérios de sucesso, pode ser necessário voltar à fase de planejamento ou análise.

O fluxograma a seguir mostra essas etapas graficamente:

11225354498 - Maria Vitória Gonçalves Vieira


Identificação da Mudança
Registro da Mudança
Rejeitada
Análise da Mudança
Mais Informações
Decisão
Aprovada
Planejamento
Implementação
Falha
Verificação
Revisão
Sucesso
Fim do Processo

Vamos detalhar um pouco mais as etapas de avaliação de impacto, aprovação e implementação de mudanças, pois elas costumam ser mais cobradas.

Avaliação de Impacto:

A avaliação de impacto envolve a análise do impacto potencial que uma mudança proposta terá em um projeto ou sistema. Isso inclui o impacto no cronograma do projeto, nos custos, nos recursos necessários, na funcionalidade do sistema, na experiência do usuário e em outros fatores. A avaliação também deve considerar o risco associado à mudança, incluindo o potencial para introduzir novos bugs ou problemas de desempenho.

Aprovação de Mudanças:

Após a avaliação de impacto, a mudança proposta é submetida para aprovação. Em muitas organizações, isso é feito por um Comitê de Controle de Mudanças (CCM), que é um grupo de partes interessadas que têm a autoridade para aprovar ou rejeitar mudanças. O CCM irá considerar a avaliação de impacto e quaisquer outros fatores relevantes, como as prioridades do negócio e a estratégia geral do projeto, ao tomar sua decisão.

Implementação de Mudanças:

Se a mudança for aprovada, o próximo passo é a implementação. Isso envolve a realização das alterações necessárias no sistema. Dependendo da natureza da mudança, isso pode envolver a codificação de novas funcionalidades, a correção de bugs, a alteração da arquitetura do sistema, a modificação da documentação, entre outras. Durante a implementação, é importante seguir todas as práticas de desenvolvimento de software relevantes para garantir que a mudança seja implementada corretamente. Isso pode incluir a realização de revisões de código, a execução de testes automatizados e a documentação adequada da mudança.

Após a implementação, a mudança deve ser verificada para garantir que foi implementada corretamente e que está funcionando conforme esperado. Qualquer problema descoberto durante esta fase deve ser abordado e corrigido. Além disso, a mudança deve ser monitorada após a implementação para identificar e resolver qualquer problema que possa surgir.

11225354498 - Maria Vitória Gonçalves Vieira

Auditoria e Controle de GCM

Auditoria de GCM é uma função importante para garantir que os processos de Gestão de Configuração e Mudanças estão sendo seguidos corretamente. O objetivo da auditoria de GCM é verificar se os itens de configuração no sistema estão em conformidade com a documentação e as especificações.

A auditoria envolve a revisão de registros para garantir que todas as alterações sejam devidamente documentadas e autorizadas, a verificação da presença de itens de configuração no sistema e a comparação com as especificações, e a verificação da conformidade com as políticas e procedimentos de GCM.

O resultado de uma auditoria de GCM pode ser um relatório detalhado indicando quaisquer problemas encontrados e recomendações para resolver esses problemas. A auditoria pode ser realizada internamente ou por um terceiro independente.

Controle de GCM

O controle de GCM envolve a monitoração e ajuste do processo de GCM para garantir que ele continue a atender às necessidades da organização. Abrange a análise de métricas de GCM, como o número de mudanças implementadas, o tempo médio para processar mudanças, o número de problemas encontrados durante as auditorias de GCM, entre outros.

As responsabilidades do controle de GCM incluem o ajuste dos processos de GCM com base em feedback e análise de métricas, a garantia de que as políticas e procedimentos de GCM estejam atualizados e a orientação da equipe em relação a práticas de GCM.

Tanto a auditoria quanto o controle de GCM desempenham papéis importantes para garantir que o processo de GCM seja eficaz e beneficie a organização. Eles ajudam a manter a qualidade e a estabilidade do sistema, ao mesmo tempo que permitem mudanças e inovações.

Git

Git é um sistema de controle de versão distribuído gratuito e de código aberto projetado para lidar com vários tipos de arquivos em diferentes projeto com

11225354498 - Maria Vitória Gonçalves Vieira

velocidade e eficiência. Foi criado por Linus Torvalds em 2005 para o desenvolvimento do kernel do Linux.

Principais características:

- Controle de Versão Distribuído: Diferentemente dos sistemas de controle de versão centralizados, o Git permite que cada desenvolvedor tenha uma cópia completa do histórico de desenvolvimento em sua máquina local. Isso significa que a maioria das operações pode ser feita localmente sem a necessidade de uma conexão com a rede. isso torna o Git muito rápido e permite que os desenvolvedores trabalhem offline.
- Integridade dos Dados: Git usa uma estrutura de dados chamada sistema de arquivos de objetos que garante a integridade criptográfica de todo o projeto. Cada arquivo e commit é verificado e identificado por uma soma de verificação SHA-1. Isso significa que é impossível alterar os conteúdos de qualquer arquivo ou diretório sem que o Git saiba.
- Branching e Merging: Git oferece funcionalidades para criação de branches (ramificações) e merging (fusão de arquivos). Isso permite que desenvolvedores trabalhem em diferentes funcionalidades ou bugs simultaneamente sem afetar o código principal. O processo de Merging costuma ser relativamente fácil no Git.
- Colaboração: Git foi projetado para ser uma ferramenta que suporta um modelo de desenvolvimento que dá a cada desenvolvedor uma sensação de trabalho simultâneo e paralelo. Isso aumenta a produtividade da equipe e permite um fluxo de trabalho mais flexível.
- Suporte à Comunidade e Popularidade: Git é usado por milhões de projetos em todo o mundo e tem uma grande comunidade de desenvolvedores que contribuem para a sua manutenção e desenvolvimento. Muitas ferramentas e serviços foram criados em torno do Git, incluindo GitHub, GitLab e Bitbucket, que fornecem interfaces de usuário amigáveis e serviços adicionais como hospedagem de repositório, integração contínua e entrega contínua (CI/CD), gerenciamento de projetos, entre outros.

Os Três Estados do Git

O Git tem um modelo de manipulação de dados que pode ser pensado como três "áreas" ou "estágios" principais, que são:

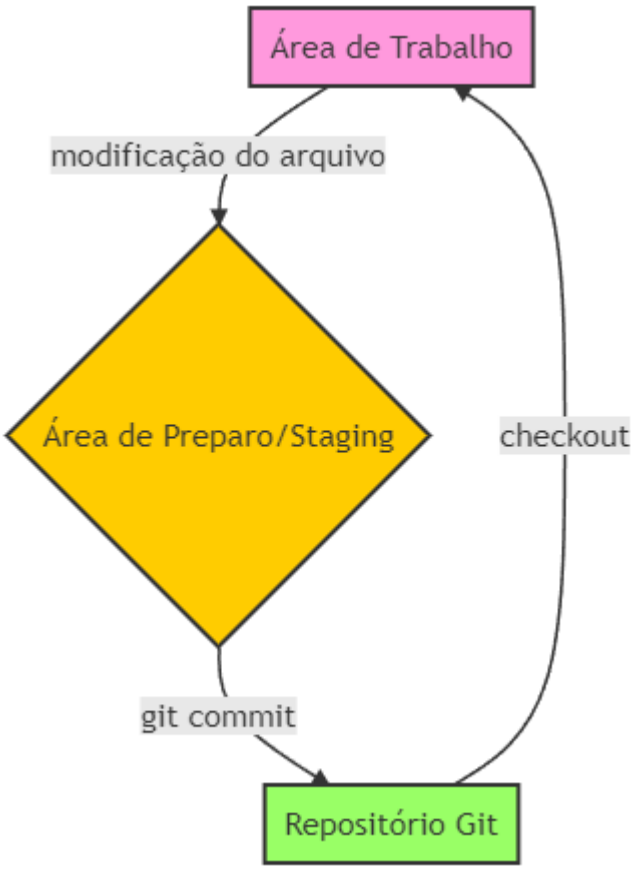
- Modified (Modificado): Neste estágio, as alterações foram feitas nos arquivos, mas ainda não foram preparadas para commit. Isso significa que o Git reconheceu que o arquivo foi alterado no diretório de trabalho, mas as alterações ainda não foram

11225354498 - Maria Vitória Gonçalves Vieira

confirmadas para serem permanentes no repositório. Essas alterações ainda não estão na área de preparo (staging area).

- Staged (Preparado): Neste estágio, o Git sabe que você modificou os arquivos e quer incluir essas modificações em seu próximo commit. Você informa ao Git que quer incluir as atualizações em um determinado arquivo para o próximo commit adicionando esses arquivos à área de preparo. O comando 'git add' é usado para adicionar um arquivo da área de trabalho para a área de preparo. As alterações neste estágio não serão "commitadas" até que o usuário decida fazê-lo.
- Commit (Committed): Este estágio representa os dados que foram salvos ou "commitados" no diretório .git, que é o diretório onde o Git armazena metadados e histórico do objeto para o repositório. Uma vez que um usuário commita suas alterações, todas as modificações são armazenadas neste local. Esta é a versão final do projeto e o Git a usa para saber o que exatamente mudou desde o último commit.

Veja esse fluxo:



11225354498 - Maria Vitória Gonçalves Vieira

O arquivo .gitignore

O arquivo .gitignore é um arquivo de texto que diz ao Git quais arquivos ou diretórios ignorar no projeto. Ou seja, arquivos listados no .gitignore não serão rastreados pelo Git e não aparecerão na lista de arquivos que foram modificados ou não estão sendo rastreados.

Este é um recurso útil para evitar que arquivos desnecessários (como arquivos temporários criados pelo sistema operacional ou pela IDE, arquivos de log, arquivos binários, senhas, chaves API, etc.) sejam incluídos em seus repositórios.

Veja alguns exemplos de como um .gitignore pode ser usado:

```
# Ignora todos os arquivos .log
*.log

# Ignora todos os arquivos na pasta /temp
/temp/*

# Ignora o arquivo secrets.txt
secrets.txt

# Mas não ignore o arquivo debug.log, mesmo se ignoramos todos os .log acima
!debug.log

# Ignora todos os arquivos .txt no diretório direto docs/, mas não em subdiretórios
de docs/
/docs/*.txt
```

Você pode ter um arquivo .gitignore diferente em cada diretório do seu projeto, com cada arquivo aplicando-se ao diretório em que está e seus subdiretórios. Para projetos maiores, é comum ter um arquivo .gitignore na raiz do projeto, que se aplica a todo o projeto.

Comandos Git

Comandos são instruções que interagem com o sistema de controle de versão do Git. Cada comando do Git tem uma função específica que permite aos desenvolvedores realizar tarefas como criar um novo repositório, fazer alterações, revisar o histórico de alterações, entre outros.

A tabela a seguir resume os principais comandos git:

11225354498 - Maria Vitória Gonçalves Vieira

Comando	Descrição
Criar novo repositório	git init
Verificar estado dos arquivos/diretórios	git status
Adicionar um arquivo em específico (staged area)	git add meu_arquivo.txt
Adicionar um diretório em específico	git add meu_diretorio
Adicionar todos os arquivos/diretórios	git add .
Adicionar um arquivo que esta listado no .gitignore (geral ou do repositório)	git add -f arquivo_no_gitignore.txt
Comitar um arquivo	git commit meu_arquivo.txt
Comitar vários arquivos	git commit meu_arquivo.txt meu_outro_arquivo.txt
Comitar informando mensagem	git commit meuarquivo.txt -m "minha mensagem de commit"
Remover arquivo	git rm meu_arquivo.txt
Remover diretório	git rm -r diretorio
Exibir histórico	git log
Exibir histórico com diff das duas últimas alterações	git log -p -2
Exibir resumo do histórico (hash completa, autor, data, comentário e quantidade de alterações (+/-))	git log --stat
Exibir informações resumidas em uma linha (hash completa e comentário)	git log --pretty=oneline



11225354498 - Maria Vitória Gonçalves Vieira

Exibir histórico com formatação específica (hash abreviada, autor, data e comentário)	git log --pretty=format:"%h - %an, %ar : %s" %h: Abreviação do hash; %an: Nome do autor; %ar: Data; %s: Comentário.
Exibir histórico de um arquivo específico	git log -- <caminho_do_arquivo>
Exibir histórico de um arquivo específico que contêm uma determinada palavra	git log --summary -S<palavra> [<caminho_do_arquivo>]
Exibir histórico modificação de um arquivo	git log --diff-filter=M -- <caminho_do_arquivo>
Exibir revisão e autor da última modificação de uma bloco de linhas	git blame -L 12,22 meu_arquivo.txt
Desfazendo alteração local (working directory)[1]	git checkout -- meu_arquivo.txt
Desfazendo alteração local (staging area)[2]	git reset HEAD meu_arquivo.txt
Exibir os repositórios remotos	git remote git remote -v
Vincular repositório local com um repositório remoto	git remote add origin git@github.com:usuario/repositorio
Exibir informações dos repositórios remotos	git remote show origin
Renomear um repositório remoto	git remote rename origin curso-git
Desvincular um repositório remoto	git remote rm curso-git
Enviar arquivos/diretórios para o repositório remoto	git push -u origin master

[1] Este comando deve ser utilizando enquanto o arquivo não foi adicionado na staged area.

[2] Este comando deve ser utilizando quando o arquivo já foi adicionado na staged area.

Listar configurações	git config --list
Setar usuário	git config --global user.name "nome"
Setar editor	git config --global core.editor vim
Setar ferramenta de merge	git config --global merge.tool vimdiff
Atualizar os arquivos no branch atual	git pull
Buscar as alterações, mas não aplicá-las no branch atual	git fetch
Clonar um repositório remoto já existente	git clone git@<caminho_do_arquivo>
Usa busca binária para encontrar o commit que introduziu um bug	git-bisect

Github

GitHub (*https://github.com*) é um serviço de hospedagem de repositórios Git baseado na web que facilita o controle de versão e a colaboração. Embora Git seja um sistema de controle de versão que pode ser usado localmente, o GitHub fornece uma interface fácil de usar e baseada na web, e adiciona muitos recursos que facilitam o trabalho colaborativo em projetos de todos os tamanhos.

Veja algumas funcionalidades úteis do GitHub:

- Colaboração em tempo real: GitHub facilita o trabalho de equipes no mesmo projeto. Desenvolvedores podem colaborar em projetos de qualquer lugar do mundo.
- Contribuição para projetos de código aberto: O site hospeda milhões de repositórios de código aberto. Qualquer pessoa pode contribuir para esses projetos ou usar o código para os seus próprios projetos.
- Rastreamento de Problemas: GitHub possui um sistema de rastreamento de problemas que permite aos usuários relatar bugs ou solicitar novos recursos. Isso facilita a organização de tarefas e a colaboração em equipe.

11225354498 - Maria Vitória Gonçalves Vieira

- Documentação: O serviço permite que os repositórios incluam arquivos de documentação, como READMEs, para explicar o projeto. Isso torna mais fácil para os outros entenderem o propósito de um projeto e como contribuir para ele.
- Integração com outras ferramentas: O repositório pode ser integrado com muitas outras ferramentas de desenvolvimento, como sistemas de integração contínua (CI/CD), e pode automatizar o teste e a implantação de seu código.
- Revisão de Código e Pull Requests: O GitHub introduziu a ideia de Pull Requests, que é uma maneira de propor mudanças que podem ser revisadas e discutidas antes de serem integradas ao projeto. Isso incentiva a revisão de código e a colaboração.
- GitHub Pages: Você pode hospedar sites estáticos diretamente de um repositório no GitHub. Esta é uma boa maneira de criar sites para projetos, currículos, blogs e muito mais.
- Controle de Versão: E, claro, o GitHub fornece todos os benefícios do controle de versão do Git, como a capacidade de rastrear mudanças, criar branches, fazer merge de código e muito mais.

Conceitos e Ações do GitHub

O GitHub introduziu uma série de conceitos e ações que podem ser realizadas na ferramenta para facilitar o fluxo de trabalho em um repositório Git. Vamos compreender melhor os conceitos de pull requests, issues, forks e GitHub Actions.

Claro, vamos discorrer sobre esses conceitos do GitHub:

- Pull Requests (PRs): No GitHub, uma Pull Request (PR) é a maneira de você propor modificações no código de um repositório. Você faz isso criando um "fork" do repositório, fazendo suas alterações nesse fork, e então pedindo que o mantenedor do repositório original "puxe" suas alterações para o repositório original. As PRs são locais onde revisões de código acontecem. Outros colaboradores podem comentar nas mudanças propostas, solicitar alterações, ou aprovar as modificações. Uma vez que uma PR é aprovada, as alterações podem ser mergeadas no projeto.
- Issues: Issues no GitHub são como tarefas, sugestões ou bugs relatados para um projeto. Eles são uma ótima maneira de organizar o trabalho em um projeto. Qualquer pessoa pode criar uma issue em um repositório público para relatar um bug ou sugerir novos recursos. As issues podem ser atribuídas a colaboradores, ter etiquetas anexadas para fácil organização, e serem referenciadas em PRs.
- Forks: Um "fork" é uma cópia de um repositório original que reside na sua conta do GitHub. Isso permite que você modifique um projeto sem afetar o original. Forks são

11225354498 - Maria Vitória Gonçalves Vieira

usados frequentemente para propor mudanças no projeto original através de Pull Requests.

- GitHub Actions: GitHub Actions é uma ferramenta de automação de CI/CD (Integração Contínua / Implantação Contínua) que permite automatizar fluxos de trabalho diretamente dentro de seu repositório no GitHub. Você pode escrever fluxos de trabalho individuais para automatizar o seu processo de desenvolvimento de software. Por exemplo, você pode configurar uma ação para executar testes sempre que um novo commit é empurrado para o seu repositório, ou para implantar seu aplicativo em um servidor sempre que você fizer um novo release.

Gitlab

O GitLab é uma plataforma de DevOps completa e de código aberto que permite que desenvolvedores colaborem em projetos de software. Assim como o GitHub, o GitLab usa o Git como seu sistema de controle de versão. No entanto, o GitLab se destaca por fornecer um conjunto mais amplo de ferramentas integradas para o ciclo de vida completo do desenvolvimento de software.

Alguns pontos de comparação entre o GitHub e o GitLab:

- Instalação e Hospedagem: O GitLab oferece a possibilidade de hospedagem na nuvem (GitLab.com) semelhante ao GitHub. No entanto, uma das principais diferenças é que o GitLab também permite que você hospede seu próprio servidor GitLab em suas próprias instalações (GitLab Self-Hosted). Isso é útil para empresas que desejam manter seu código fonte e dados de projeto em seus próprios servidores por razões de segurança ou conformidade. O GitHub também oferece uma opção semelhante chamada GitHub Enterprise Server, mas o GitLab é mais conhecido por sua solução self-hosted.
- Ferramentas de DevOps integradas: Ambas as plataformas fornecem integração com várias ferramentas de terceiros. No entanto, o GitLab tem uma gama mais ampla de ferramentas de DevOps integradas, como a integração contínua / implantação (CI / CD), monitoramento de aplicativos, gerenciamento de segurança e outras coisas mais.
- Interface do usuário: A interface do usuário entre as duas plataformas é uma questão de preferência pessoal. Alguns podem achar que o GitHub tem uma interface mais limpa e fácil de usar, enquanto outros podem preferir o GitLab por seus recursos adicionais e personalização.

11225354498 - Maria Vitória Gonçalves Vieira

- Importância da Comunidade: O GitHub tem uma grande comunidade de desenvolvedores e é o lar de uma infinidade de projetos de código aberto. Isso faz do GitHub a escolha preferencial para colaboração e contribuição em projetos de código aberto.
- Visibilidade dos Projetos: No GitHub, você pode ter repositórios públicos e privados, e a mesma coisa se aplica ao GitLab. Mas o GitLab também oferece a opção de ter projetos com visibilidade "interna", que são acessíveis a qualquer usuário logado na instância do GitLab, fornecendo um nível intermediário de privacidade.

APOSTA ESTRATÉGICA

A ideia desta seção é apresentar os pontos do conteúdo que mais possuem chances de serem cobrados em prova, considerando o histórico de questões da banca em provas de nível semelhante à nossa, bem como as inovações no conteúdo, na legislação e nos entendimentos doutrinários e jurisprudenciais[3].

O Git é um sistema de controle de versão distribuído que permite aos desenvolvedores gerenciar eficientemente o histórico de versões de seus projetos de software. Desenvolvido por Linus Torvalds em 2005, o criador do Linux, o Git foi criado para aprimorar a colaboração em grandes projetos de código aberto, oferecendo velocidade e eficiência na manipulação de grandes quantidades de dados. Ele funciona armazenando "snapshots" de um projeto, onde cada commit efetuado pelo usuário registra uma imagem do estado atual de todos os arquivos no projeto. Se um arquivo não é alterado, o Git não armazena o arquivo novamente, mas sim um link para a cópia idêntica anterior já armazenada, o que economiza espaço e aumenta a eficiência.

Além de sua arquitetura distribuída, uma das principais vantagens do Git é sua flexibilidade em vários fluxos de trabalho de desenvolvimento, desde individuais até grandes equipes. Ele suporta várias formas de desenvolvimento não-linear, permitindo fácil ramificação e mesclagem de branches, essencial para promover práticas de desenvolvimento ágil. O Git também facilita o trabalho offline, permitindo que desenvolvedores façam commits, revisões e outras modificações sem uma conexão ativa com a fonte central, sincronizando as alterações quando acessível. Essas características fazem do Git uma ferramenta poderosa e adaptável para todo tipo de projeto de software, desde startups até grandes corporações tecnológicas.

[3] Vale deixar claro que nem sempre será possível realizar uma aposta estratégica para um determinado assunto, considerando que às vezes não é viável identificar os pontos mais prováveis de serem cobrados a partir de critérios objetivos ou minimamente razoáveis.

QUESTÕES ESTRATÉGICAS

Nesta seção, apresentamos e comentamos uma amostra de questões objetivas selecionadas estrategicamente: são questões com nível de dificuldade semelhante ao que você deve esperar para a sua prova e que, em conjunto, abordam os principais pontos do assunto.

A ideia, aqui, não é que você fixe o conteúdo por meio de uma bateria extensa de questões, mas que você faça uma boa revisão global do assunto a partir de, relativamente, poucas questões.

1. (FGV - 2023 - TJ-RN - Analista Judiciário - Tecnologia de Informação – Análise de Sistemas) A desenvolvedora Amélia está trabalhando no projeto AppBravo. O controle de versões do código-fonte do AppBravo é feito pelo Git. Logo após realizar algumas modificações no branch um do código-fonte do AppBravo, Amélia precisou alterar a cópia de trabalho para o branch dois. Para não perder o progresso feito no branch um, que não estava pronto para ser commitado, Amélia utilizou o comando do Git que arquiva localmente as alterações realizadas que não foram commitadas. Portanto, Amélia utilizou o comando do Git:

A) diff;

B) fetch;

C) stash;

D) reset;

E) rebase.

Comentários:

A) Incorreta. O comando diff do Git é usado para mostrar as diferenças entre commits, commit e árvore de trabalho, etc.

B) Incorreta. O comando fetch do Git é usado para baixar o conteúdo de um repositório remoto.

C) Correta. O comando stash do Git é usado para salvar temporariamente as alterações feitas na cópia de trabalho que ainda não foram commitadas. Isso permite que o desenvolvedor mude de branch sem perder essas alterações.

11225354498 - Maria Vitória Gonçalves Vieira

D) Incorreta. O comando reset do Git é usado para redefinir o HEAD atual para um estado específico.

E) Incorreta. O comando rebase do Git é usado para mover ou combinar uma sequência de commits para um novo base commit.

Gabarito: C

2. (FGV - 2023 - SEFAZ-MG - Auditor Fiscal da Receita Estadual - Tecnologia da Informação (Tarde) O comando git que deve ser usado para descartar commits que foram feitos apenas localmente e traz o repositório local para o estado do último push do repositório remoto é o

A) git reset.

B) git revert.

C) git clean.

D) git stash.

E) git discard.

Comentários:

A) Correta. O comando git reset é usado para descartar commits no repositório local. Quando usado com a opção --hard, ele descarta todas as alterações na área de trabalho desde o último commit. Quando usado com a referência origin/, ele reverterá o repositório local para o estado do último push no repositório remoto.

B) Incorreta. O comando git revert é usado para criar um novo commit que desfaz as alterações feitas em um commit específico. Ele não descarta commits locais.

C) Incorreta. O comando git clean é usado para remover arquivos não rastreados do diretório de trabalho, mas não tem efeito sobre commits.

D) Incorreta. O comando git stash é usado para salvar temporariamente alterações que você não quer commitar ainda. Ele não descarta commits locais.

E) Incorreta. Não há um comando git discard. O comando para descartar commits locais e reverter para o estado do último push é git reset --hard origin/.

Gabarito: A

3. (FGV - 2022 - TRT - 13ª Região (PB) - Técnico Judiciário - Tecnologia da Informação) O Git é um sistema de controle de versões distribuído comumente

11225354498 - Maria Vitória Gonçalves Vieira

utilizado no desenvolvimento de software. O comando do Git que pode ser utilizado para baixar o código-fonte existente de um repositório remoto é

A) add.

B) checkout.

C) clone.

D) remote.

E) status.

Comentários:

O comando do Git que pode ser utilizado para baixar o código-fonte existente de um repositório remoto é o "clone" (alternativa C).

A opção A, "add", é usada para adicionar arquivos ao índice do Git.

A opção B, "checkout", é usada para alternar entre diferentes branches ou versões do código.

A opção D, "remote", é usada para gerenciar repositórios remotos.

A opção E, "status", é usada para verificar o estado dos arquivos no repositório local.

Gabarito: C

4. (FGV - 2022 - TRT - 13ª Região (PB) - Analista Judiciário - Tecnologia da Informação) Sobre os comandos da ferramenta de controle de versões Git, é correto afirmar que

A) o comando git add altera simultaneamente os repositórios local e remoto.

B) o comando git commit envia para o repositório remoto as alterações no espaço de trabalho local.

C) o comando git status faz o download do repositório remoto para o espaço de trabalho local.

D) o comando git push somente faz o upload das alterações que já estão em um commit.

E) o comando git pull resolve automaticamente qualquer conflito entre os códigos dos repositórios local e remoto.

11225354498 - Maria Vitória Gonçalves Vieira

Comentários:

A opção A está incorreta, pois o comando git add adiciona arquivos ao índice do Git, preparando-os para serem commitados, mas não afeta diretamente o repositório remoto.

A opção B também está incorreta. O comando git commit registra as alterações no repositório local, não enviando-as automaticamente para o repositório remoto.

A opção C está incorreta. O comando git status exibe o estado atual do repositório local, mostrando informações como arquivos modificados, adicionados ou excluídos, mas não realiza o download do repositório remoto.

A opção E está incorreta. O comando git pull mescla as alterações do repositório remoto para o repositório local, mas se houver conflitos entre os códigos, será necessário resolvê-los manualmente.

Portanto, a alternativa correta é a D. O comando git push é usado para fazer o upload das alterações confirmadas (commits) do repositório local para o repositório remoto.

Gabarito: D

5. (FGV - 2022 - TRT - 16ª REGIÃO (MA) - Analista Judiciário - Tecnologia da Informação) O comando que pode ser executado na ferramenta de versionamento Git para exibir a lista dos arquivos alterados no diretório de trabalho local é

A) git fetch

B) git gc

C) git status

D) git prune

E) git remote

Comentários:

Git status mostra os arquivos alterados localmente em relação ao repositório remoto.

As outras alternativas estão incorretas:

O comando "git fetch" é usado para buscar as alterações do repositório remoto para o repositório local, mas não exibe a lista de arquivos alterados no diretório de trabalho local.

11225354498 - Maria Vitória Gonçalves Vieira

O comando "git gc" é usado para limpar e compactar o banco de dados do Git, não exibindo a lista de arquivos alterados.

O comando "git prune" é usado para remover objetos não referenciados no repositório Git, não exibindo a lista de arquivos alterados.

O comando "git remote" é usado para gerenciar repositórios remotos, não exibindo a lista de arquivos alterados.

Gabarito: C

QUESTIONÁRIO DE REVISÃO E APERFEIÇOAMENTO

A ideia do questionário é elevar o nível da sua compreensão no assunto e, ao mesmo tempo, proporcionar uma outra forma de revisão de pontos importantes do conteúdo, a partir de perguntas que exigem
Respostas subjetivas.
São questões um pouco mais desafiadoras, porque a redação de seu enunciado não ajuda na sua resolução, como ocorre nas clássicas questões objetivas.
O objetivo é que você realize uma auto explicação mental de alguns pontos do conteúdo, para consolidar melhor o que aprendeu ;)
Além disso, as questões objetivas, em regra, abordam pontos isolados de um dado assunto. Assim, ao resolver várias questões objetivas, o candidato acaba memorizando pontos isolados do conteúdo, mas muitas vezes acaba não entendendo como esses pontos se conectam.
Assim, no questionário, buscaremos trazer também situações que ajudem você a conectar melhor os diversos pontos do conteúdo, na medida do possível.
É importante frisar que não estamos adentrando em um nível de profundidade maior que o exigido na sua prova, mas apenas permitindo que você compreenda melhor o assunto de modo a facilitar a resolução de questões objetivas típicas de concursos, ok?
Nosso compromisso é proporcionar a você uma revisão de alto nível!
Vamos ao nosso questionário:

Perguntas

1. O que é Gestão de Configuração e Mudanças (GCM)?
2. O que é um Item de Configuração?
3. Quais são as principais etapas do processo de GCM?

11225354498 - Maria Vitória Gonçalves Vieira

4. Quais são alguns exemplos de ferramentas de GCM?
5. O que é controle de versão?
6. Quais são os dois modelos principais de controle de versão?
7. O que é Git e por que é amplamente utilizado?
8. Quais são os três estágios do Git?
9. O que é o arquivo .gitignore?
10. O que é GitHub e por que é útil?
11. Quais são algumas das principais funcionalidades do GitHub?
12. Como o GitLab se compara ao GitHub?
13. O que é uma pull request?
14. O que é um Issue?
15. O que é um Fork?
16. O que é Gestão de Mudanças?
17. O que envolve a avaliação de impacto de uma mudança?
18. O que é uma Auditoria de Configuração?
19. O que é Controle de Mudanças?
20. O que é uma ferramenta de Integração Contínua (CI) e dê alguns exemplos?

Perguntas e Respostas

1. O que é Gestão de Configuração e Mudanças (GCM)?
   Resposta: A GCM é um processo de engenharia de software que visa controlar e documentar mudanças e variações em software e sistemas.
2. O que é um Item de Configuração?
   Resposta: Um item de configuração é qualquer componente de um sistema que está sujeito a mudanças e que precisa ser gerenciado para efetivamente controlar o sistema como um todo.
3. Quais são as principais etapas do processo de GCM?
   Resposta: Identificação, controle de versão, controle de mudança, auditoria de configuração e relatório de status.
4. Quais são alguns exemplos de ferramentas de GCM?
   Resposta: Git, SVN, Mercurial, Jira, Bugzilla, Redmine, Jenkins, Travis CI, CircleCI, GitHub Actions, Ansible, Chef, Puppet, Terraform, Artifactory, Nexus.
5. O que é controle de versão?
   Resposta: O controle de versão é um sistema que registra as alterações feitas em um arquivo ou conjunto de arquivos ao longo do tempo para que você possa recuperar versões específicas mais tarde.

11225354498 - Maria Vitória Gonçalves Vieira

6. Quais são os dois modelos principais de controle de versão?
   Resposta: Modelo Centralizado e Modelo Distribuído.
7. O que é Git e por que é amplamente utilizado?
   Resposta: Git é um sistema de controle de versão distribuído gratuito e de código aberto que é projetado para lidar com tudo, desde pequenos a grandes projetos com velocidade e eficiência.
8. Quais são os três estágios do Git?
   Resposta: Committed, Modificado e Preparado.
9. O que é o arquivo .gitignore?
   Resposta: O arquivo .gitignore é um arquivo de texto que diz ao Git quais arquivos ou diretórios ignorar em um projeto.
10. O que é GitHub e por que é útil?
    Resposta: O GitHub é uma plataforma de hospedagem de código-fonte que fornece recursos de controle de versão do Git, além de adicionar seus próprios recursos.
11. Quais são algumas das principais funcionalidades do GitHub?
    Resposta: Pull requests, issues, forks, GitHub Actions.
12. Como o GitLab se compara ao GitHub?
    Resposta: O GitLab é semelhante ao GitHub em muitos aspectos, mas também permite que você hospede seu próprio servidor GitLab, além de fornecer uma gama mais ampla de ferramentas de DevOps integradas.
13. O que é uma pull request?
    Resposta: Uma pull request é uma proposta de alteração em um projeto de software que está hospedado em um repositório. Uma vez que a alteração é aprovada, ela pode ser mergeada no projeto.
14. O que é um Issue?
    Resposta: Issues são problemas ou melhorias sugeridas para um projeto de software que são rastreados e gerenciados no GitHub.
15. O que é um Fork?
    Resposta: Fork é uma cópia de um repositório que reside na conta do usuário e permite experimentar alterações sem afetar o projeto original.
16. O que é Gestão de Mudanças?
    Resposta: Gestão de Mudanças é o processo de controle e coordenação de alterações nos sistemas de informação e de software.
17. O que envolve a avaliação de impacto de uma mudança?
    Resposta: A avaliação de impacto envolve a análise de como uma mudança proposta afetará o sistema existente, incluindo custos, recursos, prazos e possíveis riscos.



11225354498 - Maria Vitória Gonçalves Vieira

18. O que é uma Auditoria de Configuração?
    Resposta: A Auditoria de Configuração é uma inspeção e verificação independente da conformidade dos produtos de trabalho de software e atividades com os requisitos, procedimentos e padrões especificados.
19. O que é Controle de Mudanças?
    Resposta: O Controle de Mudanças é o processo de gerenciar e controlar alterações nos requisitos de um sistema ou projeto de software.
20. O que é uma ferramenta de Integração Contínua (CI) e dê alguns exemplos?
    Resposta: A Integração Contínua (CI) é uma prática de desenvolvimento que envolve integrar o trabalho de um desenvolvedor com o do resto da equipe de desenvolvimento com frequência. Exemplos de ferramentas de CI incluem Jenkins, Travis CI, CircleCI e GitHub Actions.

Lista de Questões Estratégicas

1. (FGV - 2023 - TJ-RN - Analista Judiciário - Tecnologia de Informação – Análise de Sistemas) A desenvolvedora Amélia está trabalhando no projeto AppBravo. O controle de versões do código-fonte do AppBravo é feito pelo Git. Logo após realizar algumas modificações no branch um do código-fonte do AppBravo, Amélia precisou alterar a cópia de trabalho para o branch dois. Para não perder o progresso feito no branch um, que não estava pronto para ser commitado, Amélia utilizou o comando do Git que arquiva localmente as alterações realizadas que não foram commitadas. Portanto, Amélia utilizou o comando do Git:

A) diff;

B) fetch;

C) stash;

D) reset;

E) rebase.

2. (FGV - 2023 - SEFAZ-MG - Auditor Fiscal da Receita Estadual - Tecnologia da Informação (Tarde) O comando git que deve ser usado para descartar commits que foram feitos apenas localmente e traz o repositório local para o estado do último push do repositório remoto é o

A) git reset.

B) git revert.

C) git clean.

D) git stash.

E) git discard.

3. (FGV - 2022 - TRT - 13ª Região (PB) - Técnico Judiciário - Tecnologia da Informação) O Git é um sistema de controle de versões distribuído comumente utilizado no desenvolvimento de software. O comando do Git que pode ser utilizado para baixar o código-fonte existente de um repositório remoto é

A) add.

B) checkout.

C) clone.

D) remote.

E) status.

4. (FGV - 2022 - TRT - 13ª Região (PB) - Analista Judiciário - Tecnologia da Informação) Sobre os comandos da ferramenta de controle de versões Git, é correto afirmar que

A) o comando git add altera simultaneamente os repositórios local e remoto.

B) o comando git commit envia para o repositório remoto as alterações no espaço de trabalho local.

C) o comando git status faz o download do repositório remoto para o espaço de trabalho local.

D) o comando git push somente faz o upload das alterações que já estão em um commit.

E) o comando git pull resolve automaticamente qualquer conflito entre os códigos dos repositórios local e remoto.

5. (FGV - 2022 - TRT - 16ª REGIÃO (MA) - Analista Judiciário - Tecnologia da Informação) O comando que pode ser executado na ferramenta de versionamento Git para exibir a lista dos arquivos alterados no diretório de trabalho local é

A) git fetch

B) git gc

C) git status

D) git prune

E) git remote

11225354498 - Maria Vitória Gonçalves Vieira

Gabaritos

1. C
2. A
3. C
4. D
5. C

11225354498 - Maria Vitória Gonçalves Vieira

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. 

Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. 

Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. 

Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. 

Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. 

Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. 

Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. 

Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. 

O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.